

New Globe

VOL. 1, NO. 035

2026-03-30

COVER STORY

Hack everything without fear

We live in a golden age of open source, and it can sometimes be easy to forget the privileges that this affords us.

Drew DeVault

I'm writing this article with vim, in a terminal emulator called urxvt, listening to music with mpv, in a Sway desktop session, on the Linux kernel. Supporting this are libraries like glibc or musl, harfbuzz, and mesa. I also have the support of the AMDGPU video driver, libinput and udev, alsa and pulseaudio.

All of this is open source. I can be reading the code for any of these tools within 30 seconds, and for many of these tools I already have their code checked out somewhere on my filesystem. It gets even better, though: these projects don't just make their code available - they accept patches, too! Why wouldn't we take advantage of this tremendous opportunity?

I often meet people who are willing to contribute to one project, but not another. Some people will shut down when they're faced with a problem that requires them to dig into territory that they're unfamiliar with. In Sway, for example, it's often places like libinput or mesa. These tools might seem foreign and scary - but to these people, at some point, so did Sway. In reality these codebases are quite accessible.

Getting around in an unfamiliar repository can be a little intimidating, but do it enough times and it'll become second nature. The same tools like gdb work just as well on them. If you have a stacktrace for a segfault originating in libinput, compile libinput with symbols and gdb will show you the file name and line number of the problem. Go there and read the code! Learn how to use tools like git grep to find stuff. Run git blame to see who wrote a confusing line of code, and send them an email! When you find the problem, don't be afraid to send a patch over instead of working around it in your own code. This is something every programmer should be comfortable doing often.

Even when the leads you're chasing down are written in unfamiliar programming languages or utilize even more unfamiliar libraries, don't despair. All programming languages

have a lot in common and huge numbers of resources are available online. Learning just enough to understand (and fix!) a particular problem is very possible, and something I find myself doing it all the time. You don't have to be an expert in a particular programming language to invoke trial & error.

If you're similarly worried about the time investment, don't be. You already set aside time to work your problem, and this is just part of that process. Yes, you'll probably be spending your time differently from your expectations - more reading code than writing code. But how is that any less productive? The biggest time sink in this process is all the time you spend worrying about how much time it's going to take, or telling me in IRC you can't solve your problem because you're not good enough to understand mesa or the kernel or whatever.

An important pastime of the effective programmer is reading and understanding the tools you use. You should at least have a basic idea of how everything on your system works, and in the places your knowledge is lacking you should make it your business to study up. The more you do this, the less scary foreign code will become, and the more productive you will be. No longer will you be stuck in your tracks because your problem leads you away from the beaten path!

{

FEATURES

A Blog, If You Can Keep It

Fly.io Blog

A boldfaced lede like this was a sure sign you were reading a carefully choreographed EffortPost from our team at Fly.io. We're going to do less of those. Or the same amount but more of a different kind of post. Either way: launch an app on Fly.io today!

Over the last 5 years, we've done pretty well for ourselves writing content for Hacker News. And that's mostly been good for us. We don't do conventional marketing, we don't have a sales team, the rest of social media is atomized over 5 different sites. Writing pieces that HN takes seriously has been our primary outreach tool.

There's a recipe (probably several, but I know this one works) for charting a post on HN:

Write an EffortPost, which is to say a dense technical piece over 2000 words long; within that rubric there's a bunch of things that are catnip to HN, including runnable code, research surveys, and explainers. (There are also cat-repellants you learn to steer clear of.)

Assiduously avoid promotion. You have to write for the audience. We get away with sporadically including a call-to-action block in our posts, but otherwise: the post should make sense even if an unrelated company posted it after you went out of business.

Pick topics HN is interested in (it helps if all your topics are au courant for HN, and we've been very lucky in that regard).

Like 5-6 more style-guide things that help incrementally. Probably 3 different teams writing for HN will have 3 different style guides with only like 1/2 overlap. Ours, for instances, instructs writers to swear.

I like this kind of writing. It's not even a chore. But it's become an impediment for us, for a couple reasons: the team serializes behind an "editorial" function here, which keeps us from publishing everything we want; worse, caring so much about our track record leaves us noodling on posts interminably (the poor Tigrises have been waiting for months for me to publish the piece I wrote about them and FoundationDB; take heart, this post today means that one is coming soon).

But worst of all, I worried incessantly about us wearing out our welcome. To my mind, we'd have 1, maybe 2 bites at the HN apple in a given month, and we needed to make them count.

That was dumb. I am dumb about a lot of things! I came around to understanding this after Kurt demanded I publish my blog post about BFAAS (Bash Functions As A Service), 500 lines of Go code that had generated 4500 words in my draft. It was only after I made the decision to stop gate-keeping this blog that I realized Simon Willison has been

disproving my "wearing out the welcome" theory, day in and day out, for years. He just writes stuff about LLMs when it interests him. I mean, it helps that he's a better writer than we are. But he's not wasting time choreographing things.

Back in like 2009, we had a blog at another company I was at. That blog drove a lot of business for us (and, on three occasions, almost killed me). It was not in the least bit optimized for HN. I like pretending to be a magazine feature writer, but I miss writing dashed-off pieces every day and clearing space for other people on the team to write as well.

So this is all just a heads up: we're trying something new. This is a very long and self-indulgent way to say "we're going to write a normal blog like it's 2008", but that's how broken my brain is after years of having my primary dopaminergic rewards come from how long Fly.io blog posts stay on the front page: I have to disclaim blogging before we start doing it, lest I fail to meet expectations.

Like I said. I'm real dumb. But: looking forward to getting a lot more stuff out on the web for people to read this year!

}

{

pkg.go.dev is more concerned with Google's interests than good engineering

Drew DeVault

pkg.go.dev sucks. It's certainly prettier than godoc.org, but under the covers, it's a failure of engineering characteristic of the Google approach.

Go is a pretty good programming language. I have long held that this is not attributable to Google's stewardship, but rather to a small number of language designers and a clear line of influences which is drawn entirely from outside of Google — mostly from Bell Labs. pkg.go.dev provides renewed support for my argument: it has all the hallmarks of Google crapware and none of the deliberate, good engineering work that went into Go's design.

It was apparent from the start that this is what it would be. pkg.go.dev was launched as a closed-source product, justified by pointing out that godoc.org is too complex to run on an intranet, and pkg.go.dev has the same problem. There are many problems to take apart in this explanation: the assumption that the only reason an open source platform is desirable is for running it on your intranet; the unstated assumption that such complexity is necessary or agreeable in the first place; and the systemic erosion of the existing (and simple!) tools which could have been used for this purpose prior to this change. The attitude towards open source was only changed following pkg.go.dev's harsh reception by the community.

But this attitude did change, and it is open-source now ¹ ², so let's give them credit for that. The good intentions are spoiled by the fact that pkg.go.dev fetches the list of modules from proxy.golang.org : a closed-source proxy through which all of your go module fetches are being routed and tracked (oh, you didn't know? They never told you, after all). Anyway, enough of the gross disregard for the values of open source and user privacy; I do have some technical problems to talk about.

One concern comes from a blatant failure to comprehend the fundamentally decentralized nature of git hosting. Thankfully, git.sr.ht is supported now ⁴ — but only the git.sr.ht, i.e. the hosted instance, not the software. pkg.go.dev hard-codes a list of centralized git hosting services, and completely disregards the idea of git hosting as software rather than as a platform . Any GitLab instance other than gitlab.com (such as gitlab.freedesktop.org or salsa.debian.org); any Gogs or Gitea like Codeberg ; cgit instances like git.kernel.org ; none of these are going to work unless every host is added and the list is kept up-to-date manually. Your intranet instance of cgit? Not a chance.

They were also given an opportunity here to fix a long-standing problem with Go package discovery, namely that it requires every downstream git repository host has to (1) provide a web interface and (2) include Go-specific meta tags in the HTML. The hubris to impose your programming language's needs onto a language-agnostic version control system! I asked: they have no interest in the better-engi-

neered — but more worksome — approach of pursuing a language agnostic design.

The worldview of the developers is whack, the new site introduces dozens of regressions, and all it really improves upon is the visual style — which could trivially have been done to godoc.org. The goal is shipping a shiny new product — not engineering a good solution. This is typical of Google's engineering ethos in general. pkg.go.dev sucks, and is added the large (and growing) body of evidence that Google is bad for Go.

id="fn:1">

Setting aside the fact that the production pkg.go.dev site is amended with closed-source patches. ↩

id="fn:2">

The GitHub comment explaining the change of heart included a link to a Google Groups discussion which requires you to log in with a Google account in order to read . ³ If you go the long way around and do some guesswork searching the archives yourself, you can find it without logging in. ↩

id="fn:3">

Commenting on Go patches also requires a Google account, by the way. ↩

id="fn:4">

But not hg.sr.ht! ↩

}

{

Free software licenses explained: MIT

Drew DeVault

This is the first in a series of posts I intend to write explaining how various free and open source software licenses work, and what that means for you as a user or developer of that software. Today we'll look at the MIT license, also sometimes referred to as the X11 or Expat license.

The MIT license is:

Both free software and open source

Permissive (and thus non-copyleft and non-viral)

This means that the license upholds the four essential freedoms of free software (the right to run, copy, distribute, study, change and improve the software) and all of the terms of the open source definition (largely the same). Further more, it is classified on the permissive/copyleft spectrum as a permissive license, meaning that it imposes relatively few obligations on the recipient of the license.

The full text of the license is quite short, so let's read it together:

The MIT License (MIT)

Copyright (c)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The first paragraph of the license enumerates the rights which you, as a recipient of the software, are entitled to. It's this section which qualifies the license as free and open source software (assuming the later sections don't disqualify it). The key grants are the right to "use" the software (freedom 0), to "modify" and "merge" it (freedom 1), and to

"distribute" and "sell" copies (freedoms 2 and 3), "without restriction". We also get some bonus grants, like the right to sublicense the software, so you could, for instance, incorporate it into a work which uses a less permissive license like the GPL.

All of this is subject to the conditions of paragraph two, of which there is only one: you must include the copyright notice and license text in any substantial copies or derivatives of the software. Thus, the MIT license requires attribution. This can be achieved by simply including the full license text (copyright notice included) somewhere in your project. For a proprietary product, this is commonly hidden away in a menu somewhere. For a free software project, where the source code is distributed alongside the product, I often include it as a comment in the relevant files. You can also add your name or the name of your organization to the list of copyright holders when contributing to MIT-licensed projects, at least in the absence of a CLA. 1

The last paragraph sets the expectations for the recipient, and it is very

important. This disclaimer of warranty is ubiquitous in nearly all free and

open source software licenses. The software is provided "as is", which is to

say, in whatever condition you found it in, for better or worse. There is no

expectation of warranty (that is to say, any support you receive is from the

goodwill of the authors and not from a contractual obligation), and there is no

guarantee of "merchantability" (that you can successfully sell it), fitness for

a particular purpose (that you can successfully use it to solve your problem),

or noninfringement (such as with respect to relevant patents). That last detail

may be of particular importance: the MIT license disclaims all liability for

patents that you might infringe upon by using the software. Other licenses often

address this case differently, such as Apache 2.0.

MIT is a good fit for projects which want to impose very few limitations on the use or reuse of the software by others. However, the permissibility of the license permits behaviors you might not like, such as creating a proprietary commercial fork of the software and selling it to others without supporting upstream. Note that the right to sell the software is an inalienable requirement of the free software

}

{

Status update, February 2020

Drew DeVault

Today I thought it'd try out something new: I have an old family recipe simmering on the stove right now, but instead of beef I'm trying out impossible beef. It cooked up a bit weird — it doesn't brown up in the same way I expect of ground beef, and it made a lot more fond than I expected. Perhaps the temperature is too high? We'll see how it fares when it's done. In the meanwhile, let's get you up to speed on my free software projects.

First, big thanks to everyone who stopped by to say “hello” at FOSDEM! Putting faces to names and getting to know everyone on a personal level is really important to me, and I would love FOSDEM even if that was all I got out of it. Got a lot of great feedback on the coming plans for SourceHut and aerc, too.

That aside, what's new? On the Wayland scene, the long-promised Sway 1.3^W1.4 was finally released, and with it wlroots 0.10.0. I've been talking it up for a while, so I won't bore you by re-listing all of the cool stuff in this release - it'll suffice to say that I think you'll enjoy it. The related tools — swayidle, swaylock, swaybg — all saw releases around the same time. The other release this month was scdoc 1.10.1, which was a simple patch release. Beyond releases, there's been some Wayland development work as well: wev received a simple bugfixes, and casa's OpenGL-based renderer rewrite has been completed nicely.

aerc progresses nicely this month as well, thanks to the support of its many dedicated contributors. Many bugfixes have landed, alongside contextual configuration options — so you can have different config settings, for example, when you have an email selected whose subject line matches a regex. A series of notmuch patches should be landing soon as well. himitsu has also seen slow progress — this pace being deliberate, as this is security-sensitive software. Several bugs have been fixed in the existing code, but there are a few more to address still. imrsh also had a little bit of improvement this month, as I started down the long road towards properly working UTF-8 support.

SourceHut improvements have also landed recently. I did some work shoring up our accessibility standards throughout the site, and SourceHut is now fully complaint with the WCAG accessibility guidelines. We now score 100% on standard performance, accessibility, and web standards compliance tests. SourceHut is the lightest weight, most usable forge. I recently fixed a bug report from a Lynx, user, too ☺ In terms of feature development, the big addition this month is support for attaching files to annotated git tags, so you can attach binaries, PGP signatures, and so on to your releases. More cool SourceHut news is coming in the post to sr.ht-announce later today.

This month's update is a little bit light on content, I'll admit. Between FOSDEM and taking some personal time, I've had less time for work this month. However, there's another

reason: I have a new secret project which I've been working on. I intend to keep this project under wraps for a while still, because I don't want people to start using it before I know if it's going to pan out or not. This project is going to take a lot of time to complete, so I hope you'll bear with me for a while and trust that the results will speak for themselves. As always, thank you for your support, and I'm looking forward to another month of awesome FOSS work.

*aerc progresses nicely this month as well, thanks to the support of its many
dedicated contributors.*

Drew DeVault

}

{

It's not okay to pretend your software is open source

Drew DeVault

Unfortunately, I find myself writing about the Commons Clause again. For those not in the know, the Commons Clause is an addendum designed to be added to free software licenses. The restrictions it imposes (you cannot sell the software) makes the resulting franken-license nonfree. I'm not going to link to the project which brought this subject back into the discussion - they don't deserve the referral - but the continued proliferation of software using the Commons Clause gives me reason to speak out against it some more.

One of my largest complaints with the Commons Clause is that it hijacks language used by open source projects to proliferate nonfree software, and encourages software using it to do the same. Instead of being a new software license, it tries to stick itself onto other respected licences - often the Apache 2.0 license. The name, "Commons Clause", is also disingenuous, hijacking language used by respected entities like Creative Commons. In truth, the Commons Clause serves to remove software from the commons¹. Combining these problems gives you language like "Apache+Commons Clause", which is easily confused with [Apache Commons][apache-commons].

Projects using the Commons Clause have also been known to describe their license as "permissive" or "open", some even calling their software "open source". This is dishonest. FOSS refers to "free and open source software". The former, free software, is defined by the free software definition, published by GNU. The latter, open source software, is defined by the open source definition, published by the OSI. Their definitions are very similar, and nearly all FOSS licenses qualify under both definitions. These are unambiguous, basic criteria which protects developers, contributors, and users of free and open source software. These definitions are so basic, important and well-respected that dismissing them is akin to dismissing climate change.

Claiming your software is open source, permissively licensed, free software, etc, when you use the Commons Clause, is lying. These lies are pervasive among users of the Commons Clause. The page listing Redis Modules, for example, states that only software under an OSI-approved license is listed. Six of the modules there are using nonfree licenses, and antirez seems content to ignore the problem until we forget about it. They're in for a long wait - we're not going to forget about shady, dishonest, and unethical companies like Redis Labs.

I don't use nonfree software², but I'm not going to sit here and tell you not to make nonfree software. You have every right to license your work in any way you choose. However, if you choose not to use a FOSS license, you need to own up to it. Don't pretend that your software is something it's not. There are many benefits to being a member of the free software community, but you are not entitled to them if your software isn't. This behavior has to stop.

Finally, I have some praise to offer. Dgraph was briefly licensed under Apache plus the Commons Clause, and had the sort of misleading and false information this article decries on their marketing website, docs, and so on. However, they've rolled it back, and Dgraph is now using the Apache 2.0 license with no modifications. Thank you!

id="fn:1">

This is why I often refer to it as the "Anti-Commons Clause", though I felt that was a bit too Stallman-esque for this article. [apache-commons]: <http://commons.apache.org/> ↩

id="fn:2">

Free as in freedom, not as in free beer. ↩

}

{

History will not remember us fondly

Drew DeVault

Today, we recall the Middle Ages as an unenlightened time (quite literally, in fact). We view the Middle Ages with a critical eye towards its brutality, lack of individual freedoms, and societal and technological regression. But we rarely turn that same critical lens on ourselves to consider how we'll be perceived by future generations. I expect the answer, upsetting as it may be, is this: the future will think poorly of us.

We possess the resources and production necessary to provide every human being on Earth with a comfortable living: adequate food, housing, health, and happiness. We have decided not to do so. We have achieved what one may consider the single unifying goal of the entire history of humanity: we have eliminated natural scarcity for our basic resources. We have done this, and we choose to deny our fellow humans their basic needs, in the cruel pursuit of profit. We have more empty homes than we have homeless people. America alone throws away enough food to feed the entire world population. And we choose to let our peers die of hunger and exposure.

We are politically destitute. Profits again drive everything — in the United States, Citizens United gave corporations unfettered access to buy and sell political will, and in the time since they have successfully installed politicians favorable to the elite class. Our corporations possess obscene wealth, coffers that rival those of nation-states, and rule over our people via their proxies in political office. Princeton published a study in 2014 which showed that the opinions of the average American citizen has a statistically negligible effect on political outcomes, while the opinions of the elite can all but decide the same outcomes. Our capitalist owners have unchallenged rule over society, and they rule it with the single-minded obsession to create profit at any cost, including lives.

The US Capitol was overrun by armed seditionists yesterday. Armed seditionists, who, by the way, were radicalized on the internet. As a computer engineer, I am complicit in this radicalization. The early internet was a sea of optimism, full of enthusiasm about the growing connectivity between people which had the potential to unite humanity like never before. We early adopters felt like world citizens: making friends, collaborating, and uniting with no respect for borders or ideology. What we hadn't realized is that we were also building the most powerful tool the world has ever seen for censorship, propaganda, and radicalization.

The companies which built this technology are modern slave drivers, broadly eroding worker freedoms in the first world, and in the third world seeking to exploit the cheapest slave labor they can find. We are developing technology which facilitates the authoritarian and genocidal policies of China. Anyone who speaks out is fired, corrections are quickly issued, and a statement of unconditional support for the profit generating, population murdering thugs is

proclaimed. I speak passionately to my peers in my field, begging them to fight back, but many lack the courage, and most don't care — so long as their exorbitant paychecks keep coming in. Money, money, money. We are at one end of a process which launders money to wash off the blood. Morals are dead.

It's not just America — democracy is on the decline worldwide. A friend in France recently took to the streets to protest against the introduction of laws protecting the police from citizen oversight. Populist traitors tore the UK out of the EU, effective last week, dooming their people to economic and political destitution. The Greek economy has failed, right-wingers are passing discriminatory laws against LGBT Poles, and conservative populism has taken hold of much of Italy, just to name a few more. Social and political systems are regressing worldwide.

Source: Freedom House

Our entire society boils down to one measure: profit. We are being eaten alive by capitalism. Americans have been brainwashed into a national ethos which is defined by capitalism. In the relentless pursuit of profits, we have eroded all political and social freedoms and created a system defined by its remarkable cruelty in a time when we have access to greater wealth and resources than at any other time in history.

Perhaps future generations won't remember us after all, considering that in that same relentless pursuit of profits we are vigorously rendering the Earth uninhabitable. But, if they do live to remember us, they will remember us as a wicked, cruel, and unempathetic lot. We will be remembered in disgrace.

}

{

Plans for Redis 3.2

Antirez

I'm back from Paris, DotScale 2015 was a very interesting conference. Before leaving I was working on Sentinel in the context of the unstable branch: the work was mainly about connection sharing. In short, it is the ability of a few Sentinels to scale, monitoring many masters. Before to leave, and now that I'm back, I tried to "secure" a set of features that will be the basis for Redis 3.2. In the next weeks I'll be focusing developing these features, so I thought it's worth to share the list with you ASAP.

Geo hashing API: This work originated from Ardb, that was originally a fork of Redis (<https://github.com/yinqiwen/ardb>), and was later extracted and improved by Matt Stancliff (<https://matt.sh/redis-geo>) that ported it to Redis. Open source is cool eh? The code needs a refactoring effort since currently duplicates parts of the sorted set implementation. It is not impossible that I may also change a few things about the API, I'm currently not sure, if there is something to fix, I'll fix it. But the bottom line is: this is a great feature, now that Matt is no longer contributing to Redis, there is a huge risk to lose this work, so I'm going to do the effort of refactoring, reviewing and merging it as the first of the tasks for Redis 3.2. I think it is a very exciting addition to the Redis API.

Bloom filters: We'll get bloom filters in 3.2. I'm not sure if this will be implemented as a String type feature like HyperLogLog are, but more likely as a new special type, since I'm interested in non trivial semantics that are more easy to provide as a new type. I've many design ideas for bloom filters, but I'm pretty sure I would like to have the ability to control from the API the accuracy/space tradeoff, perhaps not in the lower level from of specifying number of bits and hash functions to use, but in a more higher level way. Another thing I would love to have in this API is the ability of the bloom filter to auto-depollute itself (using multiple rotating filters or something like that). I'll read all the available literature and decide what to do, but we'll get this feature into 3.2.

Memory PRs: there are two important PRs from RedisLabs to improve Redis memory usage. We'll get both merged.

Memory introspection command: A command that provides information about memory, like the LATENCY command but for memory usage. Hints about where is memory consumed, if its just the RSS that is high because of past peak memory usage, hints about amount of memory used by client output buffers, ability to resize the hash tables to save some memory if needed, and so forth.

Some Redis Cluster multi DC support. This will probably just be a "static" option of Cluster slaves so that they'll not take part to promotion when the master fails. In this way using CLUSTER FAILOVER TAKEOVER it will be possible to promote all the slaves in a minority partition.

}

New List type operations: A few $O(1)$ list operations like LMERGE, and $O(N)$ operations that will be normally used with N very small so that they are most of the times $O(1)$ operations, like operations to move N elements from a list to another.

AOF safety feature: <https://github.com/antirez/redis/pull/2574>

AOF rewrites optionally using an RDB preamble, so that rewriting the AOF and reloading back the content at startup is faster.

SPOP COUNT option (already implemented, 3.2 will be the first stable versions to get it)

Redis Cluster redis-trib rebalance command, in order to automatically rehash keys to end with an more homogeneous memory usage between nodes.

A few things originally planned for 3.2 were ported to 3.0 since they were safe. A recent example is ZADD with support for options like NX and XX. In general it is possible that a few more commands about existing types will be added to Redis 3.2. This is basically a Redis version which is designed to make happy people that wanted more in the API side, since for a while we focused more on the operations aspect of Redis.

About the ETA, the work is starting Monday, and I hope they'll not take more time than the end of September, when the first RC will be pushed. Once it is RC, the RC -> Stable transition time is not scheduled, it is a function of the reporting time of critical bugs. Once for a few weeks nobody notices more bad issues, we'll go stable.

I'll follow up with new blog posts about single entries listed above, like for the Geo hashing thing, the bloom filter final implementation and API description, and so forth.

In the meantime, have fun with Redis 3.0! Comments

{

How to make your downstream users happy

Drew DeVault

There are a number of things that your FOSS project can be doing which will make the lives of your downstream users easier, particularly if you're writing a library or programmer-facing tooling. Many of your downstreams (Linux distros, pkgsrc, corporate users, etc) are dealing with lots of packages, and some minor tweaks to your workflow will help them out a lot.

The first thing to do is avoid using any build system or packaging system which is not the norm for your language. Also avoid incorporating information into your build which relies on being in your git repo — most packagers prefer to work with tarball snapshots, or to fetch your package from e.g. PyPI. These two issues are definitely the worst offenders. If you do have to use a custom build system, take your time to document it thoroughly, so that users who run into problems are well-equipped to address them. The typical build system or packaging process in use for your language already addressed most of those edge cases long ago, which is why we like it better. If you must fetch, say, version information from git, then please add a fallback, such as an environment variable.

Speaking of environment variables, another good one to support is `SOURCE_DATE_EPOCH`, for anything where the current date or time is incorporated into your build output. Many distros are striving for reproducible builds these days, which involves being able to run a build twice, or by two independent parties, and arrive at an identical checksum-verifiable result. You can probably imagine some other ways to prevent issues here — don't incorporate the full path to each file in your logs, for instance. There are more recommendations on the website linked earlier.

Though we don't like to rely on it as part of the formal packaging process, a good git discipline will also help us with the informal parts. You may already be using git tags for your releases — consider putting a changelog into your annotated tags (git tag -a). If you have good commit discipline in your project, then you can easily use git shortlog to generate such a changelog from your commit messages. This helps us understand what we can expect when upgrading, which helps incentivize us to upgrade in the first place. In *How to fuck up software releases*, I wrote about my semver tool, which you may find helpful in automating this process. It can also help you avoid forgetting to do things like update the version number somewhere in the code.

In short, to make your downstreams happy:

Don't rock the boat on builds and packaging.

Don't expect your code to always be in a git repo.

Consider reproducible builds.

Stick a detailed changelog in your annotated tag — which is easy if you have good commit discipline.

Overall, this is pretty easy stuff, and good practices which pay off in other respects as well. Here's a big "thanks" in advance from your future downstreams for your efforts in this regard!

Speaking of environment variables, another good one to support is `SOURCE_DATE_EPOCH`, for anything where the current date or time is incorporated into your build output.

Drew DeVault

}

{

A great alternative is rarely fatter than what it aims to replace

Drew DeVault

This is not always true, but in my experience, it tends to hold up. We often build or evaluate tools which aim to replace something kludgy^Wvenerable. Common examples include shells, programming languages, system utilities, and so on. Rust, Zig, etc, are taking on C in this manner; so too does zsh, fish, and oil take on bash, which in turn takes on the Bourne shell. There are many examples.

All of these tools are fine in their own respects, but they have all failed to completely supplant the software they're seeking to improve upon. 1 What these projects have in common is that they expand on the ideas of their predecessors, rather than refining them. A truly great alternative finds the nugget of truth at the center of the idea, cuts out the cruft, and solves the same problem with less.

This is one reason I like Alpine Linux, for example. It's not really aiming to replace any distro in particular so much as it competes with the Linux ecosystem as a whole. Alpine does this by being simpler than the rest: it's the only Linux system I can fit more or less entirely in my head. Compare this to the most common approach: "let's make a Debian derivative!" It kind of worked for Ubuntu, less so for everyone else. The C library Alpine ships, musl libc, is another example: it aims to replace glibc by being leaner and meaner, and I've talked about its success in this respect before.

Go is a programming language which has done relatively well in this respect. It aimed to fill a bit of a void in the high-performance internet infrastructure systems programming niche, 2 3 and it is markedly simpler than most of the other tools in its line of work. It takes the opportunity to add a few innovations — its big risk is its novel concurrency model — but Go balances this with a level of simplicity in other respects which is unchallenged among its contemporaries, 4 and a commitment to that simplicity which has endured for years. 5

There are many other examples. UTF-8 is a simple, universal approach which smooths over the idiosyncrasies of the encoding zoo which pre-dates it, and has more-or-less rendered its alternatives obsolete. JSON has almost completely replaced XML, and its grammar famously fits on a business card. 6 On the other hand, when zsh started as a superset of bash, it crippled its ability to compete on "having less warts than bash".

Rust is more vague in its inspirations, and does not start as a superset of anything. It has, however, done a poor job of scope management, and is significantly more complex than many of the languages it competes with, notably C and Go. For this reason, it struggles to root out the hold-outs in those domains, and it suffers for the difficulty in porting it to new platforms, which limits its penetration into a lot of domains that C is still thriving in. However, it succeeds in being

much simpler than C++, and I expect that it will render C++ obsolete in the coming years as such. 7

In computing, we make do with a hodge podge of hacks and kludges which, at best, approximate the solutions to the problems that computing presents us. If you start with one such hack as the basis of a supposed replacement and build more on top of it, you will inherit the warts, and you may find it difficult to rid yourself of them. If, instead, you question the premise of the software, interrogate the underlying problem it's trying to solve, and apply your insights, plus a healthy dose of hindsight, you may isolate what's right from what's superfluous, and your simplified solution just might end up replacing the cruft of yore.

id="fn:1">

Some of the listed examples have not given up and would prefer that I say something to the effect of "but the jury is still out" here. ↩

id="fn:2">

That's a lot of adjectives! ↩

id="fn:3">

More concisely, I think of Go as an "internet programming language", distinct from the systems programming languages that inspired it. Its design shines especially in this context, but its value-add is less pronounced for other tasks in the systems programming domain - compilers, operating systems, etc. ↩

id="fn:4">

The Go spec is quite concise and has changed very little since Go's inception. Go is also unique among its contemporaries for (1) writing a spec which (2) supports the development of multiple competing implementations. ↩

id="fn:5">

Past tense, unfortunately, now that Go 2 is getting stirred up. ↩

id="fn:6">

It is possible that JSON has achieved too much success in this respect, as it has found its way into a lot of use-cases for which it is less than ideal. ↩

id="fn:7">

Despite my infamous distaste for Rust, long-time readers will know that where I have distaste for Rust, I have passionate scorn for C++. I'm quite glad to see Rust taking it on, and I hope very much that it succeeds in this respect. ↩

}

{

Writing a Wayland Compositor, Part 1: Hello wlroots

Drew DeVault

This is the first in a series of many articles I'm writing on the subject of building a functional Wayland compositor from scratch. As you may know, I am the lead maintainer of sway, a reasonably popular Wayland compositor. Along with many other talented developers, we've been working on wlroots over the past few months. This is a powerful tool for creating new Wayland compositors, but it is very dense and difficult to understand. Do not despair! The intention of these articles is to make you understand and feel comfortable using it.

Before we dive in, a quick note: the wlroots team is starting a crowdfunding campaign today to fund travel for each of our core contributors to meet in person and work for two weeks on a hackathon. Please consider contributing to the campaign !

You must read and comprehend my earlier article, An introduction to Wayland, before attempting to understand this series of blog posts, as I will be relying on concepts and terminology introduced there to speed things up. Some background in OpenGL is helpful, but not required. A good understanding of C is mandatory. If you have any questions about any of the articles in this series, please reach out to me directly via sir@cmpwn.com or to the wlroots team at #sway-devel on [irc.freenode.net](https://freenode.net).

During this series of articles, the compositor we're building will live on GitHub: Wayland McWayface. Each article in this series will be presented as a breakdown of a single commit between zero and a fully functional Wayland compositor. The commit for this article is `f89092e`. I'm only going to explain the important parts - I suggest you review the entire commit separately.

Let's get started. First, I'm going to define a struct for holding our compositor's state:

Note: `mcw` is short for McWayface. We'll be using this acronym throughout the article series. We'll set one of these aside and initialize the Wayland display for it 1 :

The Wayland display gives us a number of things, but for now all we care about is the event loop. This event loop is deeply integrated into wlroots, and is used for things like dispatching signals across the application, being notified when data is available on various file descriptors, and so on.

Next, we need to create the backend:

The backend is our first wlroots concept. The backend is responsible for abstracting the low level input and output implementations from you. Each backend can generate zero or more input devices (such as mice, keyboards, etc) and zero or more output devices (such as monitors on your desk). Backends have nothing to do with Wayland - their purpose is to help you with the other APIs you need to use

as a Wayland compositor. There are various backends with various purposes:

The `drm` backend utilizes the Linux DRM subsystem to render directly to your physical displays.

The `libinput` backend utilizes `libinput` to enumerate and control physical input devices.

The `wayland` backend creates "outputs" as windows on another running Wayland compositor, allowing you to nest compositors. Useful for debugging.

The `x11` backend is similar to the Wayland backend, but opens an `x11` window on an `x11` server rather than a Wayland window on a Wayland server.

Another important backend is the `multi` backend, which allows you to initialize several backends at once and aggregate their input and output devices. This is necessary, for example, to utilize both `drm` and `libinput` simultaneously.

wlroots provides a helper function for automatically choosing the most appropriate backend based on the user's environment:

I would generally suggest using either the Wayland or X11 backends during development, especially before we have a way of exiting the compositor. If you call `wlr_backend_autocreate` from a running Wayland or X11 session, the respective backends will be automatically chosen.

We can now start the backend and enter the Wayland event loop:

If you run your compositor at this point, you should see the backend start up and... do nothing. It'll open a window if you run from a running Wayland or X11 server. If you run it on DRM, it'll probably do very little and you won't even be able to switch to another TTY to kill it.

In order to render something, we need to know about the outputs we can render on. The backend provides a `wl_signal` that notifies us when it gets a new output. This will happen on startup and as any outputs are hotplugged at runtime.

Let's add this to our server struct:

This adds a `wl_listeners` which is signalled when new outputs are added. We also add a `wl_list` (which is just a linked list provided by `libwayland-server`) which we'll later store some state in. To be notified, we must use `wl_signal_add` :

wlroots provides a helper function for automatically choosing the most appropriate backend based on the user's environment.

light"> server.

}

{

30 Minutes With MCP and flyctl

Fly.io Blog

I wrote this post on our internal message board, and then someone asked, “why is this an internal post and not on our blog”, so now it is.

well, Cursor built

I built the most basic MCP server for flyctl I could think of. It took 30 minutes.

MCP, for those unaware, is the emerging standard protocol for connecting an LLM (or an app that drives an LLM in the cloud, like Claude Desktop) to, well, anything. The “client” in MCP is the LLM; the “server” is the MCP server and the “tools” it exports. It mostly just defines an exchange of JSON blobs; one of those JSON blobs enables the LLM to discover all the tools exported by the server.

A classic example of an MCP server is (yes, really) a Python shell. MCP publishes to (say) Claude that it can run arbitrary Python code with a tool call; not only that, says the tool description, but you can use those Python tool calls to, say, scrape the web. When the LLM wants to scrape the web with Python, it uses MCP send a JSON blob describing the Python tool call; the MCP server (yes, really) runs the Python and returns the result.

Because I have not yet completely lost my mind, I chose to expose just two flyctl commands: fly logs and fly status. Because I’m lazy, I used MCP’s stdio mode; the MCP server just captures the output of flyctl. Because someone already wrote a really good Go MCP library, this whole thing is like 90 lines of code.

We may have gotten a little lucky, because we made a decision back in 2020 to give most of our flyctl commands a json mode to make them easier to drive from automation. I use that in my MCP server. I don’t know how much of a difference it made.

It’s, uh, annoyingly useful.

I loaded it up and pointed it at unpkg. I got this, from Claude:

This appears to be a globally distributed CDN service running the unpkg JavaScript package registry, with machines strategically placed around the world to provide low-latency access. There are some stability issues with a couple of the machines, and several have experienced memory-related crashes that triggered automatic restarts.

It then recited the complete configuration of the application:

The app is running on 10 machines distributed across global regions: North America: lax (Los Angeles), atl (Atlanta), ewr (Newark) Europe: lhr (London), cdg (Paris), ams (Ams-

terdam) Asia: sin (Singapore), nrt (Tokyo), hkg (Hong Kong) South America: bog (Bogotá) Oceania: syd (Sydney)

OK. But then:

Most machines are in “passing” health check status 2 machines are in “critical” status: One in ewr (Newark) with “context deadline exceeded” error One in atl (Atlanta) with “gone” status

Multiple machines have experienced restarts Several machines show “oom_killed: true” in their event history, indicating they ran out of memory

So I asked, “try getting logs for one of the critical machines”.

The root cause appears to be memory usage - the Bun process was consuming about 3.7GB of memory (out of the 4GB allocated to the machine), and there was likely a surge in requests or memory usage that pushed it over the limit. This matches what we saw in the status report, which showed multiple instances of “oom_killed: true” in the event history for several machines.

And then it dug in:

The machine was actively serving content requests with 200 OK responses until 20:47:03 PM

At that point, it ran out of memory: Out of memory: Killed process 641 (bun) total-vm:85950964kB, anon-rss:3744352kB, file-rss:12032kB, shm-mem-rss:0kB, UID:0 pgtables:16908kB oom_score_adj:0

The process was killed with SIGKILL

The system attempted to recover by restarting: reboot: Restarting system

A health check failed shortly after: Health check on port 3000 has failed. Your app is not responding properly. Services exposed on ports [80, 443] will have intermittent failures until the health check passes.

The machine then restarted successfully and the service came back up: Server listening on http://localhost:3000 ...

The health check started passing again at 20:47:46 PM: Health check on port 3000 is now passing.

The machine resumed normal operations, serving requests.

I’m honestly kind of surprised how fast this was useful. It’s faster than I find problems in apps. Just by giving some LLM some tools that do basically what I’d do.

Local MCP servers are scary. I don’t like that I’m giving a Claude instance in the cloud the ability to run a native program on my machine. I think fly logs and fly status are safe, but I’d rather know it’s safe. It would be, if I was run-

}

{

Does Rust belong in the Linux kernel?

Drew DeVault

I am known to be a bit of a polemic when it comes to Rust. I will be forthright with the fact that I don't particularly care for Rust, and that my public criticisms of it might set up many readers with a reluctance to endure yet another Rust Hot Take from my blog. My answer to the question posed in the title is, of course, "no". However, let me assuage some of your fears by answering a different question first: does Hare belong in the Linux kernel?

If I should owe my allegiance to any programming language, it would be Hare. Not only is it a systems programming language that I designed myself, but I am using it to write a kernel. Like Rust, Hare is demonstrably useful for writing kernels with. One might even go so far as to suggest that I consider it superior to C for this purpose, given that I chose to write Helios in Hare rather than C, despite my extensive background in C. But the question remains: does Hare belong in the Linux kernel?

In my opinion, Hare does not belong in the Linux kernel, and neither does Rust. Some of the reasoning behind this answer is common to both, and some is unique to each, but I will be focusing on Rust today because Rust is the language which is actually making its way towards mainline Linux. I have no illusions about this blog post changing that, either: I simply find it an interesting case-study in software engineering decision-making in a major project, and that's worth talking about.

Each change in software requires sufficient supporting rationale. What are the reasons to bring Rust into Linux? A kernel hacker thinks about these questions differently than a typical developer in userspace. One could espouse the advantages of Cargo, generics, whatever, but these concerns matter relatively little to kernel hackers. Kernels operate in a heavily constrained design space and a language has to fit into that design space. This is the first and foremost concern, and if it's awkward to mold a language to fit into these constraints then it will be a poor fit.

Some common problems that a programming language designed for userspace will run into when being considered for kernelspace are:

Strict constraints on memory allocation

Strict constraints on stack usage

Strict constraints on recursion

No use of floating point arithmetic

Necessary evils, such as unsafe memory use patterns or integer overflow

The absence of a standard library, runtime, third-party libraries, or other conveniences typically afforded to userspace

Most languages can overcome these constraints with some work, but their suitability for kernel use is mainly defined by how well they adapt to them — there's a reason that kernels written in Go, C#, Java, Python, etc, are limited to being research curiosities and are left out of production systems.

As Linus recently put it, "kernel needs trump any Rust needs". The kernel is simply not an environment which will bend to accommodate a language; it must go the other way around. These constraints have posed, and will continue to pose, a major challenge for Rust in Linux, but on the whole, I think that it will be able to rise to meet them, though perhaps not with as much grace as I would like.

If Rust is able to work within these constraints, then it satisfies the ground rules for playing in ring 0. The question then becomes: what advantages can Rust bring to the kernel? Based on what I've seen, these essentially break down to two points: 1

Memory safety

I would prefer not to re-open the memory safety flamewar, so we will simply move forward with the (dubious) assumptions that memory safety is (1) unconditionally desirable, (2) compatible with the kernel's requirements, and (3) sufficiently provided for by Rust. I will offer this quote from an unnamed kernel hacker, though:

There are possibly some well-designed and written parts which have not suffered a memory safety issue in many years. It's insulting to present this as an improvement over what was achieved by those doing all this hard work.

Regarding "trendiness", I admit that this is a somewhat unforgiving turn of phrase. In this respect I refer to the goal of expanding the kernel's developer base from a bunch of aging curmudgeons writing C 2 towards a more inclusive developer pool from a younger up-and-coming language community like Rust. C is boring 3 — it hasn't really excited anyone in decades. Rust is exciting, and its community enjoys a huge pool of developers building their brave new world with it. Introducing Rust to the kernel will certainly appeal to a broader audience of potential contributors.

But there is an underlying assumption to this argument which is worth questioning: is the supply of Linux developers dwindling, and, if so, is it to such an extent that it demands radical change?

Well, no. Linux has consistently enjoyed a tremendous amount of attention from the software development community. This week's release of Linux 6.0, one of the largest Linux releases ever, boasted more than 78,000 commits by almost 5,000 different authors since 5.15. Linux has a broad developer base reaching from many different industry stakeholders and independent contributors working on the careful development and maintenance of its hundreds of subsystems. The scale of Linux development is on a level unmatched by any other software project — free software or otherwise.

}

{

My personal journey from MIT to GPL

Drew DeVault

As I got started writing open source software, I generally preferred the MIT license. I actually made fun of the “copy-left” GPL licenses, on the grounds that they are less free. I still hold this opinion today: the GPL license is less free than the MIT license - but today, I believe this in a good way.

If you haven't yet, I suggest reading the MIT license - it's very short. It satisfies the four essential freedoms guaranteed of free software :

The right to use the software for any purpose.

The right to study the source code and change it as you please.

The right to redistribute the software to others.

The right to distribute your modifications to the software.

The MIT license basically allows you to do whatever you want with the software. It's one of the most hands-off options: “here's some code, you can do anything you want with it.” I favored this because I wanted to give users as much freedom to use my software as possible. The GPL, in addition to being a much more complex tome to understand , is more restrictive. The GPL forces you to use the GPL for derivative works as well. Clearly this affords you less freedom to use the software. Obligations are the opposite of freedoms.

When I first got into open source, I was still a Windows user. As I gradually waded deeper and deeper into the free software pond, I began to use Linux more often 1 . Even once I started using Linux as my daily driver, however, it took a while still for the importance of free software to set in. But this realization is inevitable, for a programmer immersed in Linux. It radically changes your perspective when all of the software you use guarantees these four freedoms. If I'm curious about how something works, I can usually be reading the code within a few seconds. I can find the author's name and email in the git blame and shoot them some questions. And when I find a bug, I can fix it and send them a patch.

The weight of these possibilities did not occur to me immediately, instead slowly becoming evident over time. Today, this cycle is almost muscle memory. Pulling down source, grepping for files related to an itch I need to scratch, compiling and installing the modified version, and sending my work upstream - it's become second nature to me. These days, on the rare occasion that I run into some proprietary software, this all grinds to a halt. It's like miscounting the number of steps on your staircase in the dark. These moments drive the truth home: Free software is good. It's starkly better than the alternative. And copyleft defends it. Now that I've had a taste, you bet your ass I'm not going to give it up.

}

As the number of hours I've spent on FOSS projects grew from tens of hours, to hundreds, to thousands and tens of thousands, I've learned that the effort I sink into my work far outstrips the effort required to reuse my work. The collective effort of the free software community amounts to tens of millions of hours of work, which you can download at touch of a button, for free. If the people with their fingers on that button held these same ideals, we wouldn't need the GPL. The reality, however, is that we live in a capitalist world. Our socialist free software utopia is ripe for exploitation by capitalists, and they'll be rewarded for doing so. Capitalism is about enriching yourself - not enriching your users and certainly not enriching society.

Your parents probably taught you about the Golden Rule when you were young: do unto others as you would have them do unto you. The GPL is the legal embodiment of this Golden Rule: in exchange for benefiting from my hard work, you just have to extend me the same courtesy. Its the unfortunate acknowledgement that we've created a society that incentivises people to forget the Golden Rule. I give people free software because I want them to reciprocate with the same. That's really all the GPL does. Its restrictions just protect the four freedoms in derivative works. Anyone who can't agree to this is looking to exploit your work for their gain - and definitely not yours.

I don't plan on relicensing my historical projects, but my new projects have used the GPL family of licenses for a while now. I think you should seriously consider it as well.

id="fn:1">

Fun fact: the first time I used Linux was as a teenager, in order to get around the internet filtering software my parents had installed on our Windows PC at home. ↩

```
standard-article( title: [Pick Your KotlinConf Workshop by What You Want to Learn], author: [Ekaterina Volodko], source-name: [Kotlin Blog], images: (), paragraphs: ( [Hi!], [Already coming to KotlinConf in Munich? Kick off the conference with a full day of hands-on workshops on May 20.], [Not going yet? Then this is a good moment to get your KotlinConf ticket and start with a workshop that matches what you want to learn.], [Learn how to write safe, efficient asynchronous code with Kotlin coroutines and Flow – from suspending functions and structured concurrency to hot vs. cold flows and real-world streaming use cases.], [Learn how to build shared UI across Android, iOS, desktop, and web using Compose Multiplatform.], [Improve code sharing, interoperability, performance, and app quality with advanced KMP techniques.], [Build fast, scalable Spring Boot APIs using Kotlin coroutines and virtual threads without reactive complexity.], [Learn how to build AI agents in Kotlin with Koog – from quick prototypes to production-ready systems.], [Refactor existing OO code toward functional Kotlin step by step, without breaking your app.], [See you in Munich!], ), insert-map: (:), word-count: 237, edited-for-length: false, debug-mode: false, )
```

```
standard-article( title: [Status update, September 2022], author: [Drew DeVault], source-name: [Drew DeVault], images: (), paragraphs: ( [I have COVID-19 and I am halfway through my stockpile of tissues, so I'm gonna keep this status update brief.], [In Hare news, I finally put the last pieces into place to make cross compiling as easy as possible. Nothing else particularly world-shattering going on here. I have a bunch of new stuff in my patch queue to review once I'm feeling better, however, including bigint stuff – a big step towards TLS support. Unrelatedly, TLS support seems to be progressing upstream in qbe. (See what I did there?)], [powerctl is a small new project I wrote to configure power management states on Linux. I'm pretty pleased with how it turned out. It makes for a good case study on Hare for systems programming.], [In Helios, I have been refactoring the hell out of everything, rewriting or redesigning large parts of it from scratch. Presently this means that a lot of the functionality which was previously present was removed, and is being slowly re-introduced with substantial changes. The key is reworking these features to take better consideration of the full object lifecycle – creating, copying, and destroying capabilities. An improvement which ended up being useful in the course of this work is adding address space IDs (PCIDs on x86_64), which is going to offer a substantial performance boost down the line.], [Alright, time
```

```
standard-article( title: [Your vibe coded slop PR is not welcome], author: [@sam Sam Saffron], source-name: [Sam Saffron], images: (), paragraphs: ( [I agree, it is a bit of both.], [The stark alien aspect for me is the incredible competence mixed in with incredible incompetence.], [The systems know every coding language and almost every trick in the book, but they often apply the tricks in very weird and alien ways.], [Part of it is the “eagerness to please” ... eg: you asked me to do it, so I did it.], [But part is just over reliance on hacks that should not be deployed and lack of “whole system” thinking.], [Completely agree though, you need to know how to steer this tooling to get great results.], [Also, something a lot of people do not realize, you need to know when to “give up” and start from scratch. Back to your lost ship analogy, the ship often gets so lost that that trying to steer it in the right direction is both pointless and impossible, sometimes you need a reboot. In fact I would say reboots are needed a lot more frequently than I tend to use them, cause I like playing steer the ship, but it can be counter productive.], ), insert-map: (:), word-count: 188, edited-for-length: false, debug-mode: false, )
```

```
brief-group(( brief-item([Maciej Skierkowski], source-name: [Serverless Blog], [The Serverless Framework makes it super easy to identify problems with your deployed serverless applications before they impact the quality of your service])
```

```
brief-item([Diego Zanon], source-name: [Serverless Blog], [Guest author Diego Zanon writes about building a serverless notification system for browsers using the Serverless Framework and AWS IoT.])
```

```
brief-item([Philipp Müns], source-name: [Serverless Blog], [S3 server-side encryption, support for API Gateway usage plans, default exclusion of Node.js dev dependencies and more in the Serverless Framework v1.16 release.])
```

```
brief-item([Stefanie Monge], source-name: [Serverless Blog], [The official OpenWhisk plugin for Serverless now includes support for Cloudant DB and IBM Message Hub events, exporting Web Actions and more.])
```

```
))
```

```
standard-article( title: [Redis will remain BSD licensed], author: [Antirez], source-name: [Antirez], images: (), paragraphs: ( [Today a page about the new Common Clause license in the Redis Labs web site was interpreted as if Redis itself switched license. This is not the case, Redis is, and will remain, BSD licensed. However in the era of [edit] uncontrollable spreading of information, my attempts to provide the correct information failed, and I'm still seeing everywhere “Redis is no longer open source”. The reality is that Redis remains BSD, and actually Redis Labs did the right thing supporting my effort to keep the Redis core open as usually.], [What is happening instead is that certain Redis modules, developed inside Redis Labs, are now released under the Common Clause (using Apache license as a base license). This means that basically certain enterprise add-ons, instead of being completely closed source as they could be, will be available with a more permissive license.], [I think that Redis Labs Common Clause page did not provide a clear and complete information, but software companies often make communication errors, it happens. To me however, it looks more important that while running a system software business in the “cloud era” (LOL) is very challenging using an open source license, yet Redis Labs totally understood and supported the idea that the Redis core is an open source project, in the *most permissive license
```

for a nap. Bye!],), insert-map: (:), word-count: 227, edited-for-length: false, debug-mode: false,)

ever*, that is, BSD, and during the years provided a lot of funding to the project.], [The reason why certain modules developed internally at Redis Labs are switching license, is because they are added value that Redis Labs wants to be able to provide only to end users that are willing to compile and install the system themselves, or to the Redis Labs customers using their services. But it's not ok to give away that value to everybody willing to resell it. An example of such module is RediSearch: it was AGPL and is now going to be Apache + Common Clause.], [About myself, I'll keep writing BSD code for Redis. For Redis modules I'll develop, such as Disque, I'll pick AGPL instead, for similar reasons: we live in a "Cloud-poly", so it's a good idea to go forward with licenses that will force other SaaS companies to redistribute back their improvements. However this does not apply to Redis itself. Redis at this point is a 10 years collective effort, the base for many other things that we can do together, and this base must be as available as possible, that is, BSD licensed.], [We at Redis Labs are sorry for the confusion generated by the Common Clause page, and my colleagues are working to fix the page with better wording. Comments],), insert-map: (:), word-count: 439, edited-for-length: false, debug-mode: false,)

I think that Redis Labs Common Clause page did not provide a clear and complete information, but software companies often make communication errors, it happens.

Antirez

standard-article(title: [Try a Browser Sandbox! (For Free!)], author: [Peteris Kruminis (catonmat)], source-name: [Peteris Kruminis (catonmat)], images: (), paragraphs: ([TLDR: Want to browse the web safely without messing up your computer? Try a free browser sandbox at browserling.com/browse . It runs straight in your browser. No installs, no downloads.], [What's a Browser Sandbox?], [A browser sandbox is like a "browser inside a browser". It runs on another computer in the cloud, and you control it from your own screen. You get to surf the web, but the websites never touch your actual device.], [Is It Safe to Use?], [Yep! You can click on sketchy links or check out weird websites without any risk. All the dangerous stuff stays far away – on the remote computer, not yours. Even if a site tries to install a virus or download something, it won't reach your actual system.], [Can I Open Suspicious Emails Safely?], [Yes, with a browser sandbox you can open sketchy emails or attachments without danger. If the attachment contains malware, it gets trapped inside the sandbox and can't harm your real device.], [What About Testing Suspicious URLs?], [Absolutely. A browser sandbox is the safest way to test unknown URLs. It keeps malicious scripts, drive-by downloads, and tracking attempts locked away from your real system.], [Can I Use It for Digital Forensics?], [Yes, browser sandboxes are super useful for digital forensics work. Investigators can safely open phishing emails, suspicious websites, or malware links without risking their machines or leaking any data.], [Do I Need to Download Anything?], [Nope. Just open the sandbox , pick a browser, and start browsing. It's that easy. Everything runs in your

standard-article(title: [My plans at FOSDEM: SourceHut, Hare, and Helios], author: [Drew DeVault], source-name: [Drew DeVault], images: (), paragraphs: ([FOSDEM is right around the corner, and finally in person after long years of dealing with COVID. I'll be there again this year, and I'm looking forward to it! I have four slots on the schedule (wow! Thanks for arranging these, FOSDEM team) and I'll be talking about several projects. There is a quick lightning talk on Saturday to introduce Helios and tease a full-length talk on Sunday, a meetup for the Hare community, and a meetup for the SourceHut community. I hope to see you there!], [Lightning talk: Introducing Helios], [Saturday 12:00 at H.2215 (Ferrer)], [Helios is a simple microkernel written in part to demonstrate the applicability of the Hare programming language to kernels. This talk briefly explains why Helios is interesting and is a teaser for a more in-depth talk in the microkernel room tomorrow.], [Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system, manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high performance tasks. Helios uses Hare to implement a microkernel, largely inspired by seL4.], [BoF: The Hare programming language], [Saturday 15:00 at UB2.147], [Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system, manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high performance tasks.], [At

web browser via HTML5, JavaScript, and WebSockets, so there's no software setup or weird permissions needed.], [Can I Try Different Browsers?], [Totally. You can switch between Chrome, Firefox, Edge, Safari, and even older versions if you're testing an exploit that detonates in a particular browser version. This makes it useful for developers, bug bounty hunters, and cybersecurity researchers.], [Is It Free?], [There's a free browser sandbox version with limited time. If you need more time or features, then there are paid plans too. The paid plans offer longer sessions, more browsers, and even persistent browser sessions.], [What Is Browserling?], [Browserling is an online tool that gives you access to free sandboxed browsers running on remote machines. Its use cases include safe browsing, testing websites in different browsers, and opening suspicious files and PDFs.], [Who Uses Browserling?], [Millions of people! Tech experts, digital forensics teams, IT departments, schools, and even government workers use Browserling. Big companies and researchers trust it too. Especially when checking out risky sites or testing code in different browsers.], [Happy browsing!],), insert-map: (:), word-count: 430, edited-for-length: false, debug-mode: false,)

this meeting we'll sum up the state of affairs with Hare, our plans for the future, and encourage discussions with the community. We'll also demonstrate a few interesting Hare projects, including Helios, a micro-kernel written in Hare, and encourage each other to work on interesting projects in the Hare community.], [BoF: SourceHut meetup], [Saturday 16:00 at UB2.147], [SourceHut is a free software forge for developing software projects, providing git and mercurial hosting, continuous integration, mailing lists, and more. We'll be meeting here again in 2023 to discuss the platform and its community, the completion of the GraphQL rollout and the migration to the EU, and any other topics on the minds of the attendees.], [Sunday 13:00 at H.1308 (Rolin)], [Helios is a simple microkernel written in part to demonstrate the applicability of the Hare programming language to kernels. This talk will introduce the design and rationale for Helios, address some details of its implementation, compare it with seL4, and elaborate on the broader plans for the system.],), insert-map: (:), word-count: 415, edited-for-length: false, debug-mode: false,)

New Globe

Vol. 1, No. 035 · 2026-03-30

Curated and composed automatically.